3.13 Exercises

> "Never give in, never give in, never, never, never—in nothing, great or small, large or petty—never give in.
> *Winston Churchill, address at Harrow School, 1941*

Because this interactive zyBook version may have been re-ordered and hence sections renumbered, section numbers below labeled with COD refer to the original hardcopy book's section numbers.

EXERCISE 3.13.1: [5] <COD §3.2>.

(a) What is 5ED4 — 07A4 when these values represent unsigned 16-bit hexadecimal numbers? The result should be written in hexadecimal. Show your work.

EXERCISE 3.13.2: [5] <COD §3.2>.

(a) What is 5ED4 — 07A4 when these values represent signed 16-bit hexadecimal numbers stored in sign-magnitude format? The result should be written in hexadecimal. Show your work.

EXERCISE 3.13.3: [10] <COD §3.2>.

(a) Convert 5ED4 into a binary number. What makes base 16 (hexadecimal) an attractive numbering system for representing values in computers?

EXERCISE 3.13.4: [5] <COD §3.2>.

(a) What is 4365 — 3412 when these values represent unsigned 12-bit octal numbers? The result should be written in octal. Show your work.

3.13.5: [5] <COD §3.2>.

EXERCISE

(a) What is 4365 — 3412 when these values represent signed 12-bit octal numbers stored in sign-magnitude format? The result should be written in octal. Show your work.

EXERCISE 3.13.6: [5] <COD §3.2>.

(a) Assume 185 (1011 1001) and 122 (0111 1010) are unsigned 8-bit decimal integers. Calculate 185 — 122. Is there overflow, underflow, or neither?

EXERCISE 3.13.7: [5] <COD §3.2>.

(a) Assume 185 (1011 1001) and 122 (0111 1010) are signed 8-bit decimal integers stored in sign-magnitude format. Calculate 185 + 122. Is there overflow, underflow, or neither?

EXERCISE 3.13.8: [5] <COD §3.2>.

(a) Assume 185 (1011 1001) and 122 (0111 1010) are signed 8-bit decimal integers stored in sign-magnitude format. Calculate 185 — 122. Is there overflow, underflow, or neither?

EXERCISE 3.13.9: [10] <COD §3.2>.

(a) Assume 151 (1001 0111) and 214 (1101 0110) are signed 8-bit decimal integers stored in two's complement format. Calculate 151 + 214 using saturating arithmetic. The result should be written in decimal. Show your work.

EXERCISE 3.13.10: [10] <COD §3.2>.

(a) Assume 151 (1001 0111) and 214 (1101 0110) are signed 8-bit decimal integers stored in two's complement format. Calculate 151 — 214 using saturating arithmetic. The result should be written in decimal. Show your work.

EXERCISE 3.13.11: [10] <COD §3.2>.

(a) Assume 151 (1001 0111) and 214 (1101 0110) are unsigned 8-bit integers. Calculate 151 + 214 using saturating arithmetic. The result should be written in decimal. Show your work.

EXERCISE 3.13.12: [20] <COD §3.3>.

(a) Using a table similar to that shown in COD Figure 3.6 (Multiply example using algorithm in COD Figure 3.4), calculate the product of the octal unsigned 6-bit integers 62 and 12 using the hardware described in COD Figure 3.3 (First version of the multiplication hardware). You should show the contents of each register on each step.

EXERCISE 3.13.13: [20] <COD §3.3>.

(a) Using a table similar to that shown in COD Figure 3.6 (Multiply example using algorithm in COD Figure 3.4), calculate the product of the hexadecimal unsigned 8-bit integers 62 and 12 using the hardware described in COD Figure 3.5 (Refined version of the multiplication hardware). You should show the contents of each register on each step.

EXERCISE 3.13.14: [10] <COD §3.3>.

(a) Calculate the time necessary to perform a multiply using the approach given in COD Figure 3.3 (First version of the multiplication hardware) and 3.4 (The first multiplication algorithm, using the hardware shown in COD Figure 3.3) if an integer is 8 bits wide and each step of the operation takes four time units. Assume that in step 1a an addition is always performed—either the multiplicand will be added, or a zero will be. Also assume that the registers have already been initialized (you are just counting how long it takes to do the multiplication loop itself). If this is being done in hardware, the shifts of the multiplicand and multiplier can be done simultaneously. If this is being done in software, they will have to be done one after the other. Solve for each case.

EXERCISE 3.13.15: [10] <COD §3.3>.

(a) Calculate the time necessary to perform a multiply using the approach described in the text (31 adders stacked vertically) if an integer is 8 bits wide and an adder takes four time units.

EXERCISE 3.13.16: [20] <COD §3.3>.

(a) Calculate the time necessary to perform a multiply using the approach given in COD Figure 3.7 (Fast multiplication hardware) if an integer is 8 bits wide and an adder takes four time units.

EXERCISE 3.13.17: [20] <COD §3.3>.

(a) As discussed in the text, one possible performance enhancement is to do a shift and add instead of an actual multiplication. Since 9×6, for example, can be written $(2 \times 2 \times 2 + 1) \times 6$, we can calculate 9×6 by shifting 6 to the left three times and then adding 6 to that result. Show the best way to calculate 0x33 × 0x55 using shifts and adds/subtracts. Assume both inputs are 8-bit unsigned integers.

EXERCISE 3.13.18: [20] <COD §3.4>.

(a) Using a table similar to that shown in COD Figure 3.10 (Division example using the algorithm in COD Figure 3.9), calculate 60 divided by 17 using the hardware described in COD Figure 3.8 (First version of the division hardware). You should show the contents of each register on each step. Assume both inputs are unsigned 6-bit integers.

EXERCISE 3.13.19: [30] <COD §3.4>.

(a) Using a table similar to that shown in COD Figure 3.10 (Division example using the algorithm in COD Figure 3.9), calculate 60 divided by 17 using the hardware described in COD Figure 3.11 (An improved version of the division hardware). You should show the contents of each register on each step. Assume A and B are unsigned 6-bit integers. This algorithm requires a slightly different approach than that shown in COD Figure 3.9 (A division algorithm, using the hardware in COD Figure 3.8). You will want to think hard about this, do an experiment or two, or else go to the web to figure out how to make this work correctly. (Hint: one possible solution involves using the fact that COD Figure 3.11 (An improved version of the division hardware) implies the remainder register can be shifted either direction.)

EXERCISE 3.13.20: [5] <COD §3.5>.

(a) What decimal number does the bit pattern `0×0C000000` represent if it is a two's complement integer? An unsigned integer?

EXERCISE 3.13.21: [10] <COD §3.5>.

(a) If the bit pattern `0x94000000` is placed into the Instruction Register, what LEGv8 instruction will be executed?

EXERCISE 3.13.22: [10] <COD §3.5>.

(a) What decimal number does the bit pattern `0×0C000000` represent if it is a floating point number? Use the IEEE 754 standard

EXERCISE 3.13.23: [10] <COD §3.5>.

(a) Write down the binary representation of the decimal number 63.25 assuming the IEEE 754 single precision format.

EXERCISE 3.13.24: [10] <COD §3.5>.

(a) Write down the binary representation of the decimal number 63.25 assuming the IEEE 754 double precision format.

EXERCISE 3.13.25: [10] <COD §3.5>.

(a) Write down the binary representation of the decimal number 63.25 assuming it was stored using the single precision IBM format (base 16, instead of base 2, with 7 bits of exponent).

EXERCISE 3.13.26: [20] <COD §3.5>.

(a) Write down the binary bit pattern to represent -1.5625×10^{-1} assuming a format similar to that employed by the DEC PDP-8 (the leftmost 12 bits are the exponent stored as a two's complement number, and the rightmost 24 bits are the fraction stored as a two's complement number). No hidden 1 is used. Comment on how the range and accuracy of this 36-bit pattern compares to the single and double precision IEEE 754 standards.

EXERCISE 3.13.27: [20] <COD §3.5>.

(a) IEEE 754-2008 contains a half precision that is only 16 bits wide. The leftmost bit is still the sign bit, the exponent is 5 bits wide and has a bias of 15, and the mantissa is 10 bits long. A hidden 1 is assumed. Write down the bit pattern to represent -1.5625×10^{-1} assuming a version of this format, which uses an excess-16 format to store the exponent. Comment on how the range and accuracy of this 16-bit floating point format compares to the single precision IEEE 754 standard.

EXERCISE 3.13.28: [20] <COD §3.5>.

(a) The Hewlett-Packard 2114, 2115, and 2116 used a format with the leftmost 16 bits being the fraction stored in two's complement format, followed by another 16-bit field which had the leftmost 8 bits as an extension of the fraction (making the fraction 24 bits long), and the rightmost 8 bits representing the exponent. However, in an interesting twist, the exponent was stored in sign-magnitude format with the sign bit on the far right! Write down the bit pattern to represent -1.5625×10^{-1} assuming this format. No hidden 1 is used. Comment on how the range and accuracy of this 32-bit pattern compares to the single precision IEEE 754 standard.

EXERCISE 3.13.29: [20] <COD §3.5>.

(a) Calculate the sum of 2.6125×10^{1} and $4.150390625 \times 10^{-1}$ by hand, assuming A and B are stored in the 16-bit half precision described in COD Exercise 3.27. Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps.

EXERCISE 3.13.30: [30] <COD §3.5>.

(a) Calculate the product of and by hand, assuming A and B are stored in the 16-bit half precision format described in COD Exercise 3.27. Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps; however, as is done in the example in the text, you can do the multiplication in human-readable format instead of using the techniques described in COD Exercises 3.12 through 3.14. Indicate if there is overflow or underflow. Write your answer in both the 16-bit floating point format described in COD Exercise 3.27 and also as a decimal number. How accurate is your result? How does it compare to the number you get if you do the multiplication on a calculator?

EXERCISE 3.13.31: [30] <COD §3.5>.

(a) Calculate by hand divided by . Show all the steps necessary to achieve your answer. Assume there is a guard, a round bit, and a sticky bit, and use them if necessary. Write the final answer in both the 16-bit floating point format described in COD Exercises 3.27 and in decimal and compare the decimal result to that which you get if you use a calculator.

EXERCISE 3.13.32: [20] <COD §3.10>.

(a) <COD §3.10> Calculate by hand, assuming each of the values is stored in the 16-bit half precision format described in COD Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

EXERCISE 3.13.33: [20] <COD §3.10>.

(a) Calculate by hand, assuming each of the values is stored in the 16-bit half precision format described in COD Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

EXERCISE 3.13.34: [10] <COD §3.10>.

(a) Based on your answers to COD Exercises 3.32 and 3.33, does ?

EXERCISE 3.13.35: [30] <COD §3.10>.

(a) Calculate by hand, assuming each of the values is stored in the 16-bit half precision format described in COD Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

EXERCISE 3.13.36: [30] <COD §3.10>.

(a) Calculate by hand, assuming each of the values is stored in the 16-bit half precision format described in COD Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

EXERCISE 3.13.37: [10] <COD §3.10>.

(a) Based on your answers to COD Exercises 3.35 and 3.36, does ?

EXERCISE 3.13.38: [30] <COD §3.10>.

(a) Calculate by hand, assuming each of the values is stored in the 16-bit half precision format described in COD Exercise 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

EXERCISE 3.13.39: [30] <COD §3.10>.

(a) Calculate by hand, assuming each of the values is stored in the 16-bit half precision format described in COD Exercises 3.27 (and also described in the text). Assume 1 guard, 1 round bit, and 1 sticky bit, and round to the nearest even. Show all the steps, and write your answer in both the 16-bit floating point format and in decimal.

EXERCISE 3.13.40: [10] <COD §3.10>.

(a) Based on your answers to COD Exercises 3.38 and 3.39, does ?

EXERCISE 3.13.41: [10] <COD §3.5>.

(a) Using the IEEE 754 floating point format, write down the bit pattern that would represent . Can you represent exactly?

EXERCISE 3.13.42: [10] <COD §3.5>.

(a) What do you get if you add to itself four times? What is ? Are they the same? What should they be?

EXERCISE 3.13.43: [10] <COD §3.5>.

(a) Write down the bit pattern in the fraction of value assuming a floating point format that uses binary numbers in the fraction. Assume there are 24 bits, and you do not need to normalize. Is this representation exact?

EXERCISE 3.13.44: [10] <COD §3.5>.

(a) Write down the bit pattern in the fraction of value assuming a floating point format that uses Binary Coded Decimal (base 10) numbers in the fraction instead of base 2. Assume there are 24 bits, and you do not need to normalize. Is this representation exact?

EXERCISE 3.13.45: [10] <COD §3.5>.

(a) Write down the bit pattern assuming that we are using base 15 numbers in the fraction of value instead of base 2. (Base 16 numbers use the symbols 0—9 and A—F. Base 15 numbers

would use 0—9 and A—E.) Assume there are 24 bits, and you do not need to normalize. Is this representation exact?

EXERCISE 3.13.46: [20] <COD §3.5>.

(a) Write down the bit pattern assuming that we are using base 30 numbers in the fraction of value instead of base 2. (Base 16 numbers use the symbols 0—9 and A—F. Base 30 numbers would use 0—9 and A—T.) Assume there are 20 bits, and you do not need to normalize. Is this representation exact?

EXERCISE 3.13.47: [45] <COD §§3.6, 3.7>.

(a) The following C code implements a four-tap FIR filter on input array `sig_in`. Assume that all arrays are 16-bit fixed-point values.

```
for(i = 3; i < 128; i++)
   sig_out[i] = sig_in[i - 3] * f[0] + sig_in[i - 2] * f[1] +
                sig_in[i - 1] * f[2] + sig_in[i] * f[3];
```

Assume you are to write an optimized implementation of this code in assembly language on a processor that has SIMD instructions and 128-bit registers. Without knowing the details of the instruction set, briefly describe how you would implement this code, maximizing the use of sub-word operations and minimizing the amount of data that is transferred between registers and memory. State all your assumptions about the instructions you use.